UNIP – UNIVERSIDADE PAULISTA

Curso de Engenharia da Computação

Gabriel Marques
Gabriel Luiz Bastos Pereira
Diego Ferraz
Pedro Henrique de Souza Lemos
Wellington Tomiyoshi Santos Koga

Banco de dados de telemetria IoT para dados críticos: plataforma de monitoramento meteorológico para a Defesa Civil de São José dos Campos

Atividade Prática Supervisionada

São José dos Campos
2026

RA N062576 – Gabriel Marques
RA G8659C1 – Gabriel Luiz Bastos Pereira
RA N050FA1 – Diego Ferraz
RA N454830 – Pedro Henrique de Souza Lemos
RA G773470 – Wellington Tomiyoshi Santos Koga

Banco de dados de telemetria IoT para dados críticos: plataforma de monitoramento meteorológico para a Defesa Civil de São José dos Campos

Atividade Prática Supervisionada apresentada ao curso de Engenharia da Computação da Universidade Paulista – UNIP, campus São José dos Campos, disciplina de Laboratório de Projeto de Banco de Dados (LPBD), como requisito parcial de avaliação.

UNIP – Universidade Paulista
Curso de Engenharia da Computação

Orientador: Prof. Fernando Mauro

São José dos Campos
2026

Resumo

Dados de monitoramento de chuva e vento usados pela Defesa Civil são dados críticos: um valor perdido, adulterado ou atrasado pode atrasar um alerta de deslizamento ou alagamento. Este trabalho apresenta o projeto e o protótipo de um banco de dados de telemetria IoT em PostgreSQL, aplicado a duas estações meteorológicas simuladas em São José dos Campos (SP), que medem temperatura e umidade do ar, chuva, direção e velocidade do vento e radiação solar a cada minuto e transmitem lotes a cada dez minutos. A partir de uma versão inicial focada em desempenho, o grupo levantou requisitos de dados críticos e implementou, no próprio banco, a validação de cada leitura com datas vindas somente do relógio do servidor, eventos imutáveis, papéis de mínimo privilégio, isolamento por cliente com *Row Level Security*, senhas com bcrypt e bloqueio por tentativas, e uma trilha de auditoria somente de inclusão encadeada por hash SHA-256. Um servidor em Node.js e um painel em HTML, CSS e JavaScript puros completam o protótipo, com atualização em tempo real, backups diários verificados e uma fila de contingência que guarda os lotes quando o banco está fora do ar. Seis meses de operação (cerca de 529 mil leituras) foram simulados e os mecanismos foram testados: a adulteração proposital da auditoria foi detectada no registro exato, o PostgreSQL foi derrubado durante o envio de dados sem perda de leituras, e uma cópia de segurança foi restaurada com as mesmas contagens e a trilha de auditoria íntegra. As consultas mantiveram os ganhos das otimizações (até 200 vezes com view materializada), e as validações aumentaram em cerca de 30 % o tempo de envio de uma leitura por vez, com impacto desprezível em lotes de dez. Conclui-se que um banco relacional bem modelado pode concentrar as garantias de integridade, confidencialidade, rastreabilidade e disponibilidade exigidas por dados críticos, sem depender da boa conduta da aplicação.

Palavras-chave: banco de dados; dados críticos; PostgreSQL; auditoria; IoT.

Abstract

Rain and wind monitoring data used by civil defense agencies are critical data: a lost, tampered or delayed value may delay a landslide or flood warning. This work presents the design and prototype of an IoT telemetry database in PostgreSQL, applied to two simulated weather stations in São José dos Campos, Brazil, which measure air temperature and humidity, rainfall, wind direction and speed, and solar radiation every minute and transmit batches every ten minutes. Starting from a performance-oriented first version, the group gathered critical-data requirements and implemented, inside the database, validation of every reading with timestamps taken only from the server clock, immutable events, least-privilege roles, per-customer isolation with Row Level Security, bcrypt passwords with lockout, and an append-only audit trail chained by SHA-256 hashes. A Node.js server and a plain HTML, CSS and JavaScript dashboard complete the prototype, with real-time updates, verified daily backups and a contingency queue that keeps batches while the database is down. Six months of operation (about 529 thousand readings) were simulated and the mechanisms were tested: deliberate tampering of the audit trail was detected at the exact record, PostgreSQL was shut down while data was being sent without losing readings, and a backup was restored with identical row counts and an intact audit trail. Queries kept the gains of the optimizations (up to 200 times with a materialized view), and validation increased single-reading inserts by about 30 %, with negligible impact on ten-reading batches.

Keywords: database; critical data; PostgreSQL; audit trail; IoT.

Sumário

1 Introdução 5
2 Fundamentação teórica 6
2.1 Banco de dados, SGBD e modelo relacional 6
2.2 Transações e durabilidade 6
2.3 Programação no servidor e desempenho 6
2.4 Dados críticos e segurança da informação 7
3 Plano de desenvolvimento, arquitetura e requisitos 8
3.1 Arquitetura 8
3.2 Metodologia de simulação e testes 9
3.3 Requisitos funcionais e não funcionais 10
4 Modelagem do software 11
4.1 Modelo de dados 12
4.2 Regras implementadas no banco 12
5 Protótipo 15
5.1 Ferramentas utilizadas 15
5.2 Servidor 15
5.3 Telas 16
6 Por que o protótipo atende a dados críticos 22
7 Resultados e discussão 24
7.1 Carga e detecção de eventos 24
7.2 Testes de segurança e continuidade 24
7.3 Desempenho e custo das validações 25
8 Conclusão 26

Referências 27

1 Introdução

Desastres hidrometeorológicos, como inundações e deslizamentos, estão entre os eventos que mais causam mortes e prejuízos no Brasil. Em 2024, o Centro Nacional de Monitoramento e Alertas de Desastres Naturais ampliou para 1.133 a lista de municípios monitorados (CEMADEN, 2024), e só nesses municípios foram registrados, em 2023, 815 eventos de inundação e 526 deslizamentos (Nakano; Maru, 2024, p. 70). No Litoral Norte paulista, vizinho ao Vale do Paraíba, os pluviômetros registraram em fevereiro de 2023 663 mm de chuva em 24 horas em Bertioga (Nachtigall, 2023).

Cabe aos municípios manter a população informada sobre áreas de risco e eventos extremos (Brasil, 2012), e para isso a Defesa Civil depende de dados que chegam continuamente de estações em campo. Esses dados são **críticos**: se uma leitura de chuva se perde, é alterada sem registro ou chega tarde, o alerta pode não acontecer. Por isso, além de rápido, o banco de dados que os recebe precisa garantir integridade, controle de acesso, rastreabilidade e continuidade.

Problema. Uma primeira versão do banco, feita pelo grupo, já recebia leituras em volume e gerava alarmes, mas não protegia o dado: todos os acessos usavam o superusuário, não havia registro de quem alterava regras, uma leitura com data inválida derrubava o lote inteiro e as partições mensais acabariam em dezembro de 2026, o que faria toda transmissão falhar a partir de janeiro.

Objetivo. Projetar e prototipar um banco de dados de telemetria IoT em PostgreSQL adequado a dados críticos, com servidor e painel web, e comprovar suas garantias com testes. Especificamente: apresentar os conceitos envolvidos; levantar requisitos e modelar o sistema; implementar no banco validação, auditoria, controle de acesso e continuidade; e medir o custo dessas garantias no desempenho.

Escopo. Foram implementados o banco de dados, um servidor em Node.js e um painel em HTML, CSS e JavaScript. O equipamento de campo e o meio de transmissão foram substituídos por um simulador que gera leituras realistas para duas estações da Defesa Civil de São José dos Campos, Centro e Zona Norte.

2 Fundamentação teórica

2.1 Banco de dados, SGBD e modelo relacional

Um **banco de dados** é uma coleção de dados relacionados, organizada para um propósito; o **sistema gerenciador de banco de dados** (SGBD) é o software que permite defini-lo, manipulá-lo e compartilhá-lo entre aplicações e usuários (Elmasri; Navathe, 2018). Em vez de cada programa tratar acesso concorrente, recuperação de falhas, integridade e autorização, o SGBD assume essas tarefas para todos (Silberschatz; Korth; Sudarshan, 2020). Essa centralização é o ponto de partida deste trabalho: uma regra que vive só na aplicação pode ser contornada por outro programa; uma regra que vive no SGBD vale para todos.

O modelo relacional representa os dados como tabelas manipuladas por operações sobre conjuntos (Codd, 1970). Sobre ele, o SGBD garante restrições declaradas no esquema: **chave primária** (no projeto, estação mais data e hora impede gravar a mesma leitura duas vezes), **chave estrangeira** (um sensor só pertence a uma estação existente) e `CHECK` (o perfil do usuário só pode ser administrador, operador ou visualizador). A tabela de leituras usa o **formato largo**, uma coluna por grandeza, que reduz linhas e simplifica consultas quando as grandezas são fixas.

2.2 Transações e durabilidade

Uma **transação** é uma unidade de trabalho com as propriedades ACID: atomicidade (tudo ou nada), consistência (de um estado válido a outro), isolamento (transações concorrentes não se enxergam pela metade) e durabilidade (o que foi confirmado sobrevive a falhas) (Härder; Reuter, 1983). A durabilidade vem do registro antecipado em log (*write-ahead logging*, WAL): o SGBD grava a alteração no log antes de confirmá-la e, após uma queda, refaz o que foi confirmado (Mohan *et al.*, 1992). No PostgreSQL, esse log também permite a recuperação para qualquer instante (PITR) (PostgreSQL Global Development Group, 2025a). No projeto, receber um lote é uma transação: leituras, eventos e alarmes entram juntos ou não entram.

2.3 Programação no servidor e desempenho

O PostgreSQL executa lógica dentro do banco em PL/pgSQL (PostgreSQL Global Development Group, 2025b): **funções** encapsulam operações (uma função `SECURITY DEFINER` roda com os privilégios de quem a criou, liberando uma operação sem liberar a

tabela); ***triggers*** rodam automaticamente a cada inclusão, alteração ou exclusão, sem que a aplicação possa evitá-los; e ***event triggers*** reagem a comandos de estrutura como `ALTER` e `DROP`.

Para o desempenho com séries temporais (Jensen; Pedersen; Thomsen, 2017; Grzesik; Mrozek, 2020), o projeto usa: **índice B-tree composto** (estação, data e hora), que no PostgreSQL 18 também serve a consultas só por período, graças ao *skip scan* (PostgreSQL Global Development Group, 2025c); **índice BRIN**, muito pequeno, para dados gravados em ordem de tempo; **particionamento mensal**, em que o banco lê só os meses consultados e move um mês inteiro para o arquivo sem copiar linhas; e **view materializada**, que guarda agregações prontas para os painéis (Chirkova; Yang, 2012).

2.4 Dados críticos e segurança da informação

Neste trabalho, **dado crítico** é aquele cuja perda, alteração indevida, exposição ou indisponibilidade pode causar dano relevante a pessoas. A segurança da informação organiza sua proteção em **confidencialidade**, **integridade** e **disponibilidade** (ABNT, 2022a), com controles como gestão de acesso, registro de eventos e cópias de segurança (ABNT, 2022b); a LGPD exige medidas técnicas que protejam dados pessoais de acessos não autorizados (Brasil, 2018). Os conceitos usados no protótipo são:

- **mínimo privilégio**: cada programa e usuário opera só com os privilégios de que precisa (Saltzer; Schroeder, 1975). No banco, isso se faz com papéis distintos e com a segurança em nível de linha (*Row Level Security*, RLS), que filtra quais linhas cada usuário vê (PostgreSQL Global Development Group, 2025d);
- **senhas com bcrypt**: função de hash propositalmente lenta e com *salt*, que dificulta ataques de força bruta (Provos; Mazières, 1999);
- **auditoria à prova de adulteração**: cada registro guarda o hash do anterior, de modo que alterar ou apagar um registro quebra a corrente e se torna detectável (Schneier; Kelsey, 1999; Haber; Stornetta, 1991);
- **RPO e RTO**: quanto dado se aceita perder e em quanto tempo o serviço deve voltar após uma falha (Swanson *et al.*, 2010);
- **segurança da aplicação web**: proteção contra controle de acesso quebrado e injeção de SQL e de scripts (XSS), as falhas mais comuns segundo a OWASP (OWASP Foundation, 2021).

3 Plano de desenvolvimento, arquitetura e requisitos

O problema foi organizado com o método 5W2H, já usado pelo grupo: o quê (receber e guardar leituras que geram alertas, com garantias de dado crítico), por quê (uma leitura perdida ou adulterada atrasa um alerta), onde (estações em campo, servidor da Defesa Civil e navegador dos operadores), quando (uma leitura por minuto e um lote a cada 10 minutos, sem parar), quem (dispositivos, operadores, órgãos de consulta e administrador), como (PostgreSQL com regras e segurança internas, servidor Node.js e painel web) e quanto (só software livre). Em seguida, a primeira versão do banco foi executada do zero e submetida a casos de borda; o Quadro 1 resume os problemas encontrados, que orientaram as melhorias. Cada melhoria foi feita numa ramificação própria no GitHub e testada recriando o banco do zero.

Quadro 1 – Problemas encontrados na primeira versão

Problema observado	**Risco para dados críticos**
Partições criadas só na carga, até dezembro de 2026	Toda transmissão falharia a partir de janeiro de 2027
Uma leitura com data inválida derrubava o lote inteiro	Uma leitura ruim descartava as outras nove
Leituras com data no futuro eram aceitas; data de recebimento informada pelo chamador	Dado com data falsa entrava no histórico
Reenvio de lote já gravado gerava falha de sensor	O log registrava falhas que não aconteceram
Todos os acessos com o superusuário `postgres`	Sem confidencialidade nem separação de funções
Sem registro de quem alterou regras ou reconheceu eventos	Um limite de alarme podia ser elevado sem rastro
Sem backup e recuperação	Uma falha de disco apagaria o histórico

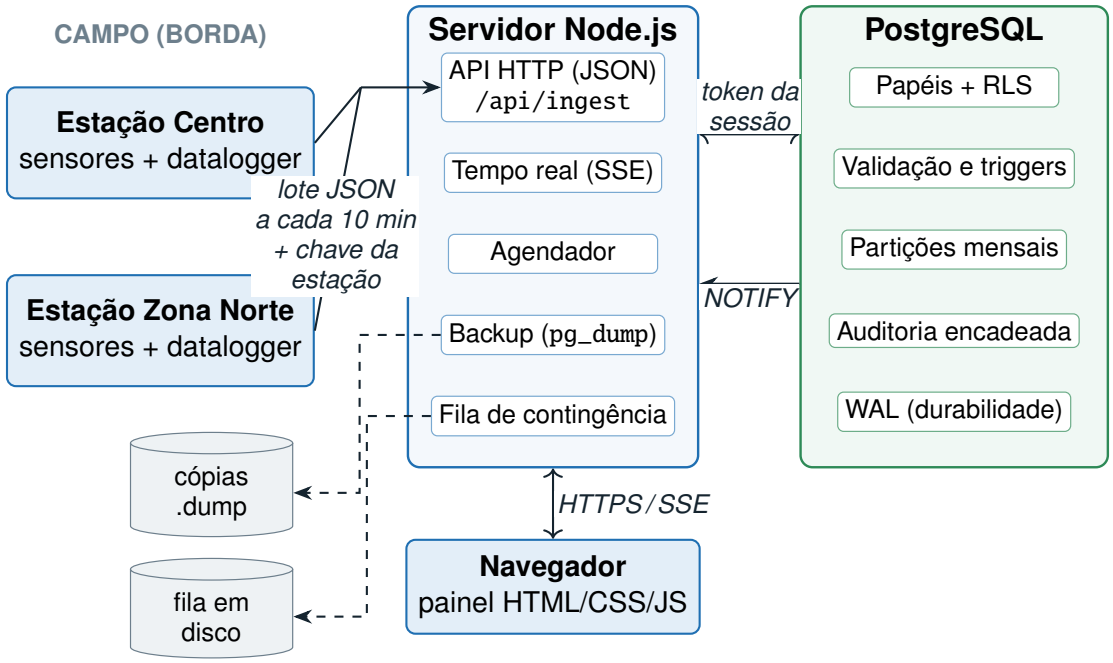
Fonte: Elaborado pelos autores (2026).

3.1 Arquitetura

A arquitetura segue as camadas sensor, borda e nuvem de Grzesik e Mrozek (2020), com um servidor de aplicação entre o banco e os usuários (Figura 1). Cada estação mede a cada minuto e envia um lote em JSON a cada dez minutos, identificado por uma chave própria. O servidor recebe os lotes, atende o painel, envia atualizações em tempo real e

executa tarefas agendadas. O banco concentra as regras: valida cada leitura, gera eventos e alarmes, controla o acesso e registra a auditoria.

Figura 1 – Arquitetura do sistema



Fonte: Elaborado pelos autores (2026), adaptado de Grzesik e Mrozek (2020).

A decisão central é que o banco **não confia na aplicação**: o servidor se conecta com papéis de privilégio mínimo e repassa apenas o *token* da sessão do usuário; é o banco que o valida e descobre quem é o usuário, seu perfil e as estações que pode ver. Mesmo com o servidor web comprometido, um invasor não se passa por outro usuário sem um *token* válido.

3.2 Metodologia de simulação e testes

Um simulador gera leituras realistas (ciclo dia e noite, sazonalidade, tardes chuvosas e um dia de temporal), com cerca de 2 % de lotes atrasados e 0,5 % de leituras com o código de erro -9999. A carga cobre seis meses; depois, o servidor continua transmitindo em tempo real. Os testes foram de segurança (tentar burlar cada regra, inclusive como superusuário), de continuidade (derrubar o banco, fazer e restaurar backups) e de desempenho, com a metodologia de Grzesik e Mrozek (2020): escrita de 1.000 pontos, um por envio e em lotes de dez, e consultas de agregação repetidas 500 vezes em quatro cenários, antes e depois das melhorias, no mesmo ambiente.

3.3 Requisitos funcionais e não funcionais

Requisitos funcionais descrevem os serviços do sistema; não funcionais restringem o sistema como um todo, como segurança e desempenho (Sommerville, 2018). Com dados críticos, os não funcionais pesam tanto quanto os funcionais: um sistema que gera o alarme certo mas perde dados numa queda não serve ao seu propósito (Pressman; Maxim, 2021). Os quadros a seguir listam os requisitos levantados.

Quadro 2 – Requisitos funcionais

ID	**Descrição**
RF01	Receber lotes de leituras em JSON, identificando a estação por chave própria.
RF02	Validar cada leitura (data ausente, inválida, futura, antiga ou de mês arquivado) sem descartar o restante do lote.
RF03	Usar somente o relógio do servidor em toda data de registro; nenhum usuário digita data.
RF04	Detectar falhas de sensor e de transmissão, calcular a chuva do dia e gerar alarmes pelas regras configuradas.
RF05	Exibir painel em tempo real, gráficos, chuva por dia, histórico e exportação em CSV.
RF06	Permitir ao operador reconhecer eventos, registrando automaticamente quem e quando.
RF07	Permitir ajustar regras de alarme e faixas dos sensores (operador) e parâmetros do banco (administrador).
RF08	Gerenciar usuários com perfis, troca de senha no primeiro acesso e desbloqueio.
RF09	Auditar alterações, logins, exportações, backups e mudanças de estrutura, com antes e depois.
RF10	Verificar a integridade da auditoria e apontar o primeiro registro adulterado.
RF11	Fazer backup diário e sob demanda, verificar cada cópia e permitir a restauração.
RF12	Arquivar meses antigos e criar com antecedência as partições dos meses seguintes.

Fonte: Elaborado pelos autores (2026).

Quadro 3 – Requisitos não funcionais

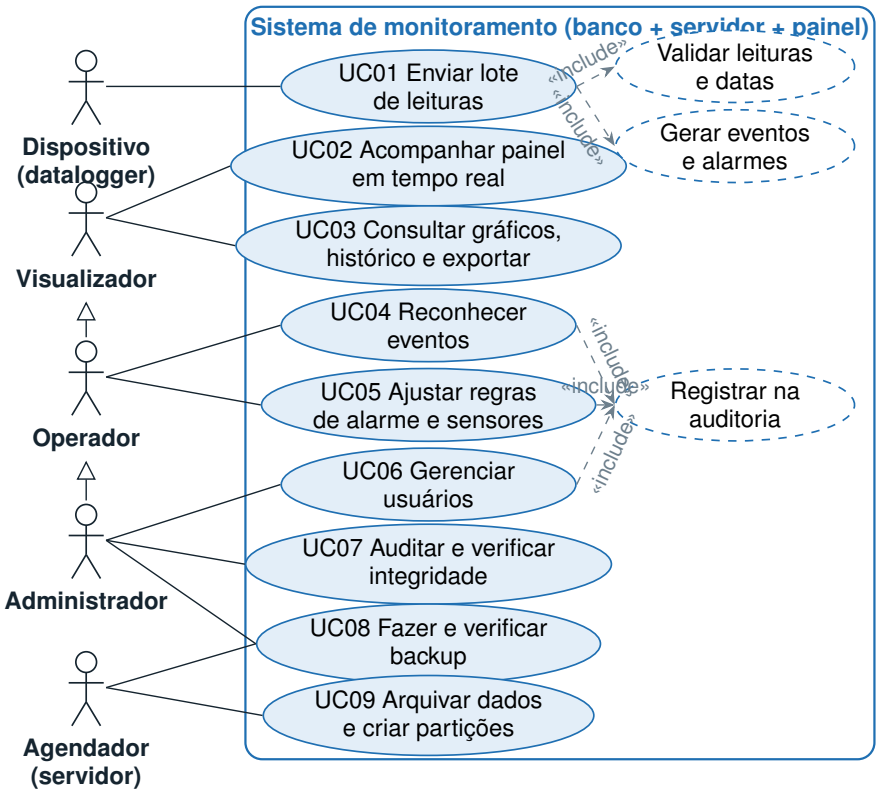
ID	**Categoria**	**Descrição**
RNF01	Integridade	Regras implementadas no banco, que nenhuma aplicação contorna; eventos imutáveis.
RNF02	Confidencialidade	Papéis de mínimo privilégio, isolamento por cliente, senhas com bcrypt, sessões guardadas só como hash.
RNF03	Rastreabilidade	Auditoria somente de inclusão, encadeada por hash, que torne detectável qualquer alteração.
RNF04	Disponibilidade	Nenhuma leitura perdida com o banco fora do ar; perda máxima de 24 h com backup diário.
RNF05	Desempenho	Painel atualizado segundos após o lote; validações sem impacto relevante no envio em lote.
RNF06	Segurança web e portabilidade	Consultas parametrizadas, proteção contra CSRF e XSS; Windows e Linux com software livre.

Fonte: Elaborado pelos autores (2026).

4 Modelagem do software

A modelagem usou a UML para casos de uso e classes (Booch; Rumbaugh; Jacobson, 2006) e um diagrama entidade-relacionamento para o banco. Os atores são o **dispositivo** (datalogger que envia os lotes), o **visualizador** (consulta e exporta), o **operador** (também reconhece eventos e ajusta alarmes), o **administrador** (também gerencia usuários, auditoria, backups e parâmetros) e o **agendador** do servidor. A Figura 2 mostra os casos de uso; a herança entre atores representa os perfis cumulativos, e os «include» mostram que todo lote é validado e toda alteração é auditada.

Figura 2 – Diagrama de casos de uso



Fonte: Elaborado pelos autores (2026).

O diagrama de classes (Figura 3) representa o domínio. Suas operações estão no próprio banco, como funções e *triggers* em PL/pgSQL: `receberLote` é `fn_receber_lote`, `reconhecer` é `fn_reconhecer_eventos` e `verificarIntegridade` é `fn_verificar_auditoria`.

Quadro 4 – Caso de uso UC01 – Enviar lote de leituras

Ator / **pré-condição**	Dispositivo; estação ativa com chave de API cadastrada.
Fluxo básico	1. O dispositivo envia o lote para `/api/ingest` com a chave no cabeçalho. 2. O banco identifica a estação pelo hash da chave e registra falha de transmissão se o lote atrasou. 3. Classifica cada leitura e separa as rejeitadas. 4. Grava as válidas e novas; os *triggers* descartam valores fora da faixa, calculam a chuva acumulada e geram alarmes. 5. Registra a transmissão e avisa os painéis em tempo real.
Alternativos e exceções	Leituras repetidas são ignoradas sem gerar eventos. Leituras inválidas geram um evento `LEITURA_REJEITADA` por motivo, e o resto do lote é gravado. Chave inválida: envio recusado e auditado. Banco fora do ar: o lote vai para a fila de contingência.
Regra	A data de recebimento é sempre o relógio do servidor.

Fonte: Elaborado pelos autores (2026).

Quadro 5 – Caso de uso UC07 – Auditar e verificar integridade

Ator / **pré-condição**	Administrador autenticado.
Fluxo básico	1. O administrador abre a auditoria; a verificação roda sozinha. 2. O banco percorre a trilha em ordem, recalcula o SHA-256 de cada registro com o hash do anterior e compara com o gravado. 3. O administrador filtra por ação, objeto, usuário e período e abre o antes e depois de cada alteração.
Alternativo	Registro alterado ou removido: a tela mostra o número do primeiro registro com problema.
Regra	A trilha é somente de inclusão; desligar essa proteção é um comando de estrutura e também é auditado.

Fonte: Elaborado pelos autores (2026).

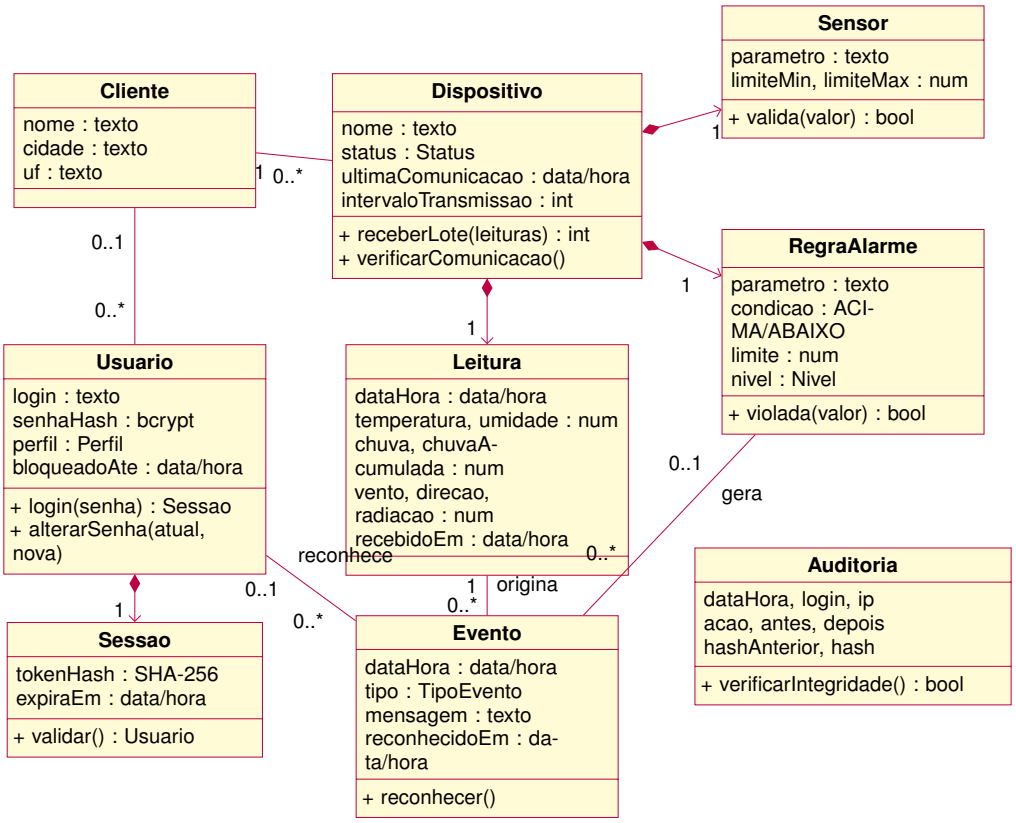
4.1 Modelo de dados

O modelo físico tem 14 tabelas (Figura 4), num diagrama gerado do catálogo do banco para corresponder exatamente ao implementado. A tabela `leitura` tem uma linha por estação por minuto, chave primária (`id_dispositivo`, `data_hora`) e partições mensais; os meses antigos são movidos inteiros para `leitura_arquivo`, com índice BRIN, e resumidos em `leitura_diaria`. Os eventos ficam numa tabela única, no formato de log, com cinco tipos: transmissão OK, falha de transmissão, falha de sensor, alarme e leitura rejeitada. As regras de alarme cadastradas são chuva acumulada acima de 50 mm (crítico), vento acima de 17 m/s, temperatura acima de 35 °C e umidade abaixo de 20 % (alerta).

4.2 Regras implementadas no banco

Todo dado entra por `fn_receber_lote`, que não tem parâmetro de data: o recebimento é sempre `now()`. Numa única instrução SQL, a função lê o JSON (valores mal formatados viram nulo, sem abortar o lote), classifica cada leitura (data no futuro além de 2 min, mais antiga que 7 dias ou de mês sem partição ativa são rejeitadas), grava um evento por motivo

Figura 3 – Diagrama de classes do domínio

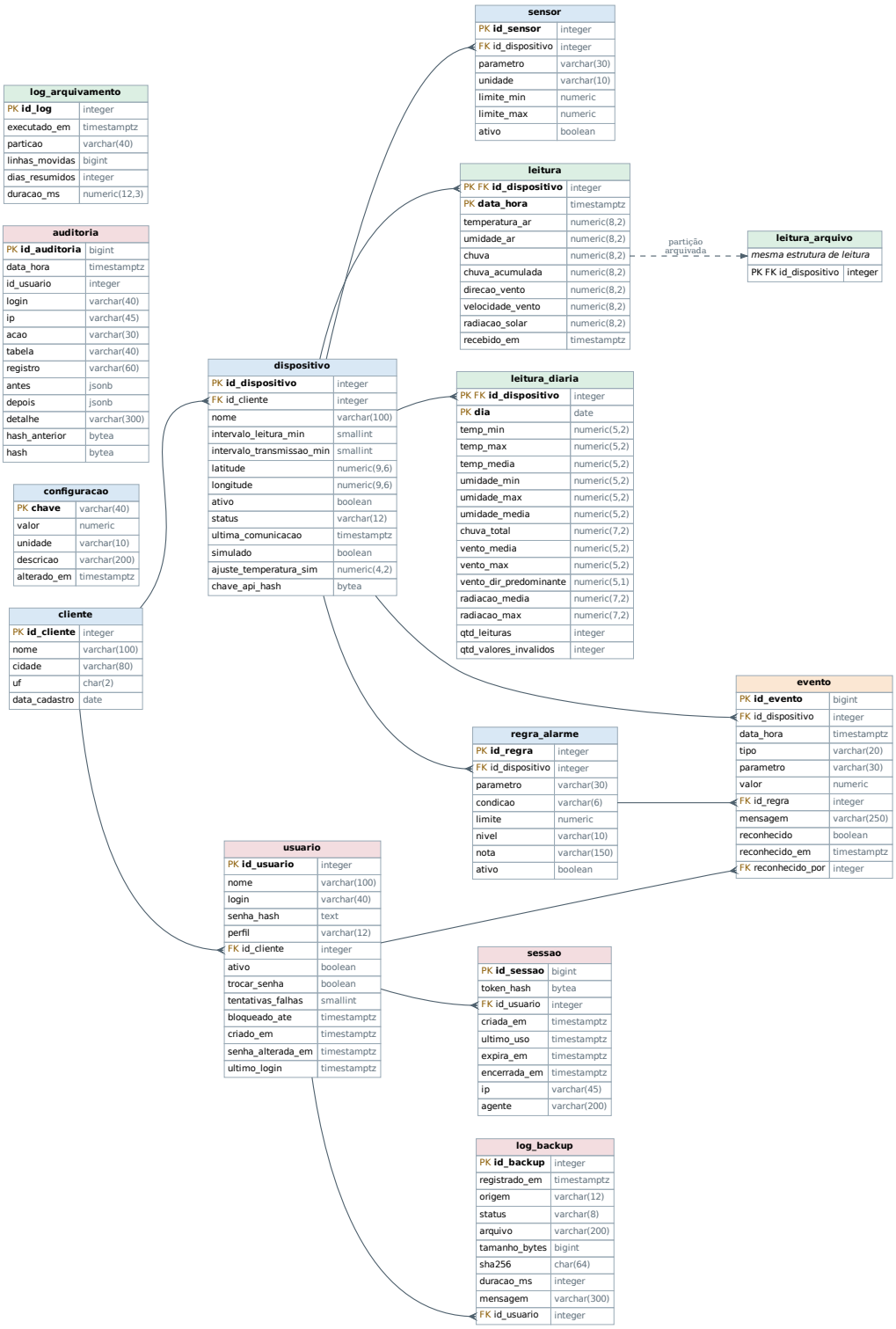


Fonte: Elaborado pelos autores (2026).

de rejeição e insere só as leituras válidas e novas, o que corrigiu as falhas de sensor "fantasmas" da primeira versão. Como defesa adicional, o *trigger* da tabela recusa data no futuro mesmo em inclusões diretas.

Na auditoria, um *trigger* define a data com o relógio do servidor, identifica o usuário pela sessão e calcula o SHA-256 do registro concatenado ao hash do anterior, sob um bloqueio que garante a ordem da corrente. Diariamente, o banco arquiva os meses antigos (`DETACH` e `ATTACH PARTITION`) e cria as partições dos dois meses seguintes.

Figura 4 – Modelo de dados (diagrama entidade-relacionamento)



Fonte: Elaborado pelos autores (2026), gerado do catálogo do PostgreSQL com Graphviz.

5 Protótipo

O protótipo é funcional: os scripts criam o banco do zero, o simulador gera seis meses de histórico e continua transmitindo em tempo real, e o painel opera sobre esses dados com usuários reais do banco. O código e as instruções de execução estão no repositório do grupo no GitHub (https://github.com/Chulalafe3/APS_Banco_de_dados).

5.1 Ferramentas utilizadas

Quadro 6 – Ferramentas utilizadas

Ferramenta	**Uso no projeto**
PostgreSQL (versões 16 a 18)	SGBD: armazenamento, regras, segurança, auditoria e recuperação
PL/pgSQL	Funções, procedimentos, *triggers* e *event triggers*
pgcrypto	bcrypt das senhas, SHA-256 da auditoria e das sessões, geração de *tokens*
pgAdmin 4 (PSQL Tool)	Execução dos scripts e consultas
Node.js com o pacote `pg`	Servidor HTTP sem *framework*: API, tempo real, agendador, backup e contingência (OpenJS Foundation, 2026)
HTML, CSS e JavaScript puros	Painel web com onze telas, gráficos em SVG próprios, sem bibliotecas externas
Server-Sent Events	Envio das atualizações do servidor ao navegador em tempo real (MDN Web Docs, 2026)
`pg_dump` e `pg_restore`	Cópias de segurança, verificação e restauração
Git e GitHub	Versionamento, uma ramificação por conjunto de melhorias
Graphviz e Playwright	Geração do diagrama do banco e captura automatizada das telas
LaTeX com abnTeX2	Redação deste trabalho nas normas da ABNT
Assistente de IA (Claude)	Apoio à revisão de código, aos testes e à redação, sob supervisão do grupo

Fonte: Elaborado pelos autores (2026).

O servidor foi escrito em Node.js por desempenho: ele apenas repassa consultas ao banco, e nesse tipo de carga, dominada por entrada e saída, o modelo assíncrono do Node.js atende muitas conexões com pouca sobrecarga (OpenJS Foundation, 2026). Para manter o código pequeno e auditável, o servidor usa só o módulo HTTP nativo e um único pacote externo, o driver `pg`. O painel antigo, feito em Python com Streamlit na primeira versão, continua funcionando sobre o mesmo banco.

5.2 Servidor

O servidor tem seis módulos:

- **banco**: mantém um conjunto de conexões para cada papel do PostgreSQL e abre cada requisição numa transação com o *token* da sessão do usuário;
- **API**: rotas para cada tela; todas as consultas são parametrizadas, o que impede injeção de SQL;
- **tempo real**: escuta os avisos do banco (`LISTEN`) e os repassa por *Server-Sent Events* só aos navegadores que podem ver aquela estação;
- **agendador**: a cada minuto verifica a comunicação das estações e esvazia a fila de contingência; a cada dez minutos simula a transmissão e atualiza a view materializada; diariamente arquiva, cria partições e faz o backup;
- **backup**: executa o `pg_dump`, calcula o SHA-256, mantém o manifesto e aplica a retenção;
- **contingência**: se o banco estiver fora do ar, grava em disco cada lote recebido e o reenvia quando o banco volta.

Na comunicação com o navegador, a sessão fica num *cookie* `HttpOnly` e `SameSite=Strict`, inacessível a scripts; toda requisição que altera dados precisa de um cabeçalho próprio do painel, o que bloqueia CSRF; e uma política de segurança de conteúdo (CSP) só permite scripts e estilos do próprio servidor (OWASP Foundation, 2021).

5.3 Telas

O painel foi desenhado para uma central de operação, em que a pergunta principal é se é preciso agir agora. A identidade visual usa a escala de avisos meteorológicos (amarelo para atenção, laranja para alerta, vermelho para crítico) sobre uma base azul-acinzentada. Em vez de `innerHTML`, toda a interface é montada com elementos criados um a um e textos inseridos como texto, de modo que um dado vindo do banco nunca é interpretado como código (proteção contra XSS). A Figura 5 mostra a tela de acesso; no primeiro acesso, o usuário é obrigado a criar uma senha própria.

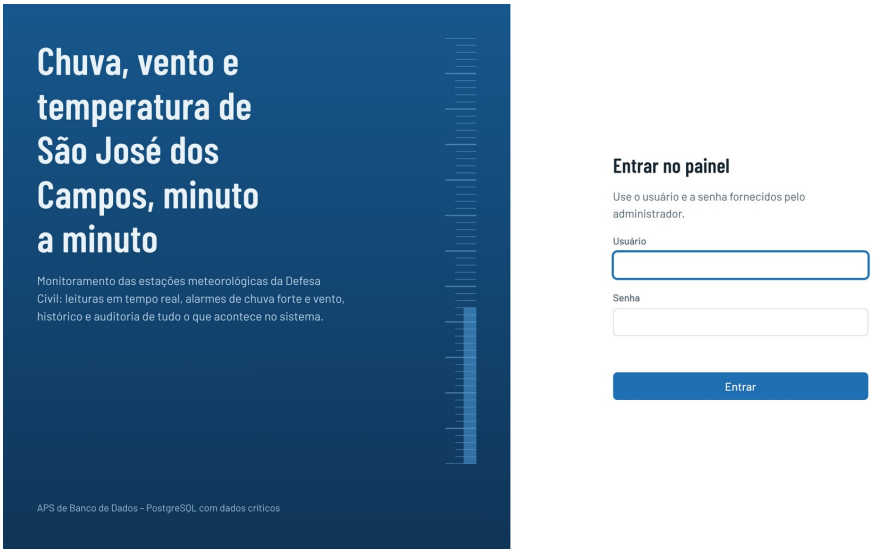
O painel (Figura 6) responde em uma frase se as estações estão normais. Cada estação tem um pluviômetro que enche com a chuva do dia até a linha do limite crítico, a temperatura com a tendência das últimas seis horas e as demais grandezas. Os números de hoje e os últimos alarmes e falhas ficam abaixo, e tudo se atualiza sozinho quando chega um lote. O relógio do topo mostra a hora do servidor, não a do computador do usuário.

O painel também tem tema escuro, próprio para salas de operação, e se adapta à tela do celular (Figura 7). No celular, o usuário mostrado é um operador, e o menu não exibe as telas exclusivas do administrador.

A tela de gráficos (Figura 8) mostra temperatura, umidade, vento e radiação em 24 horas, 7 ou 30 dias, com as linhas dos limites de alarme. Os gráficos são desenhados em SVG pelo próprio painel, sem bibliotecas externas.

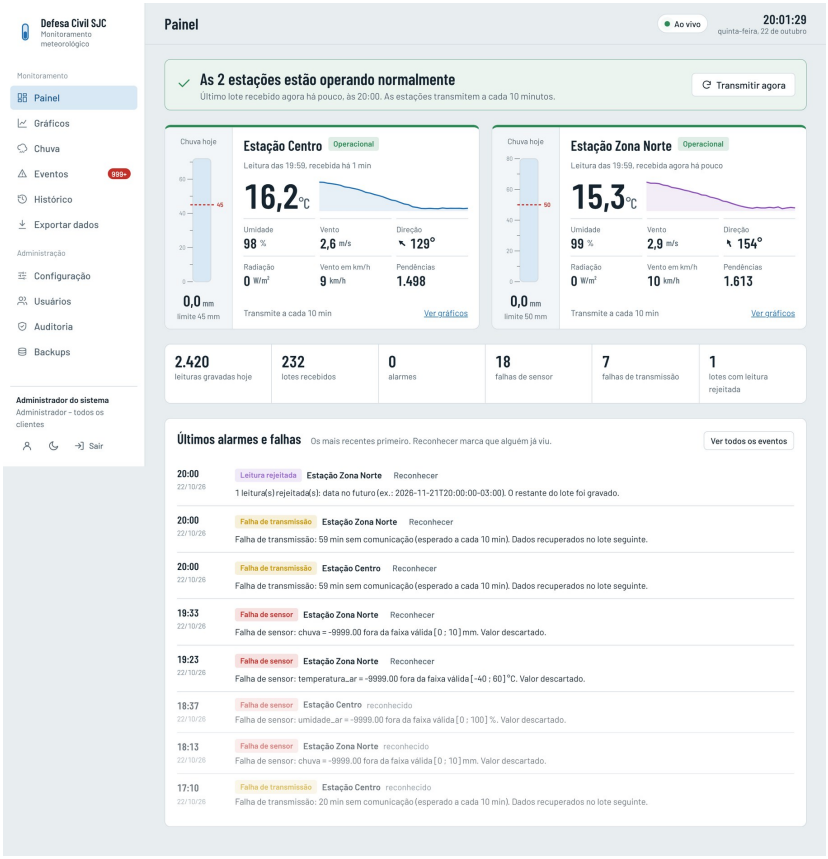
A tela de eventos (Figura 9) permite filtrar por tipo, estação e período e reconhecer

Figura 5 – Tela de acesso



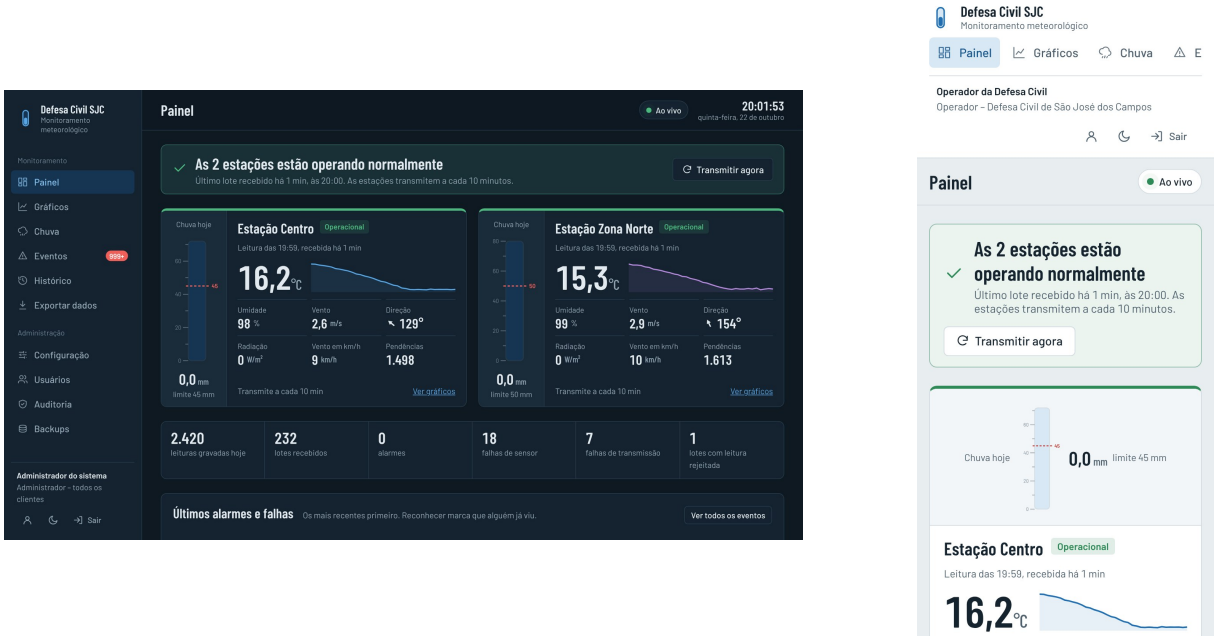
Fonte: Elaborado pelos autores (2026).

Figura 6 – Painel em tempo real



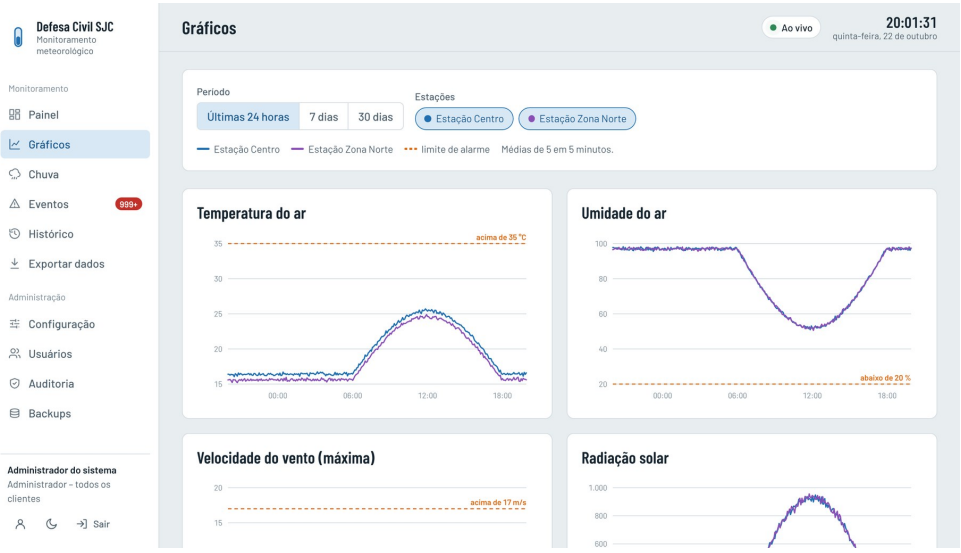
Fonte: Elaborado pelos autores (2026).

Figura 7 – Tema escuro e versão para celular



Fonte: Elaborado pelos autores (2026).

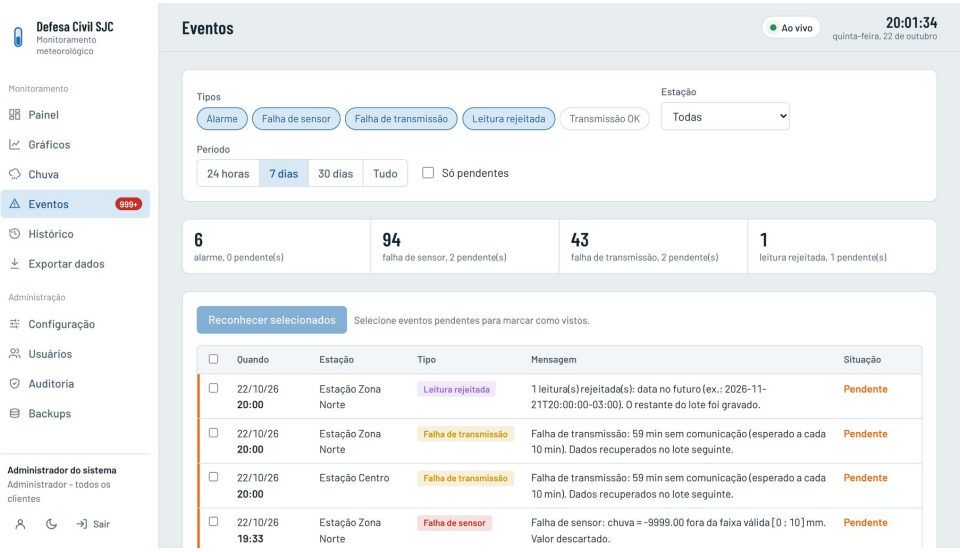
Figura 8 – Gráficos com limites de alarme



Fonte: Elaborado pelos autores (2026).

eventos em lote. Para cada evento reconhecido, mostra quem reconheceu e quando, informações preenchidas pelo banco.

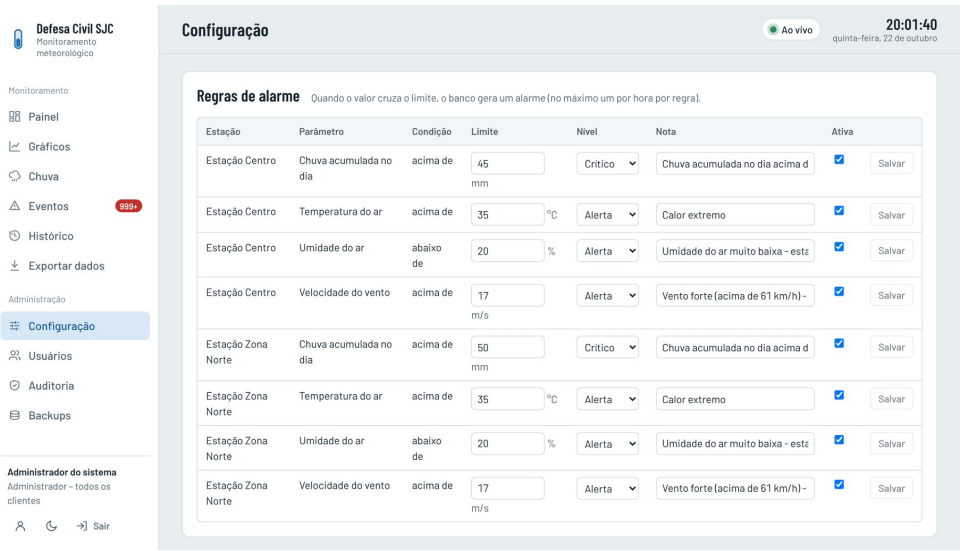
Figura 9 – Eventos e reconhecimento



Fonte: Elaborado pelos autores (2026).

A tela de configuração (Figura 10) permite ao operador ajustar as regras de alarme e as faixas dos sensores; os parâmetros de datas e retenção só podem ser alterados pelo administrador. Cada alteração vai para a auditoria com os valores de antes e depois.

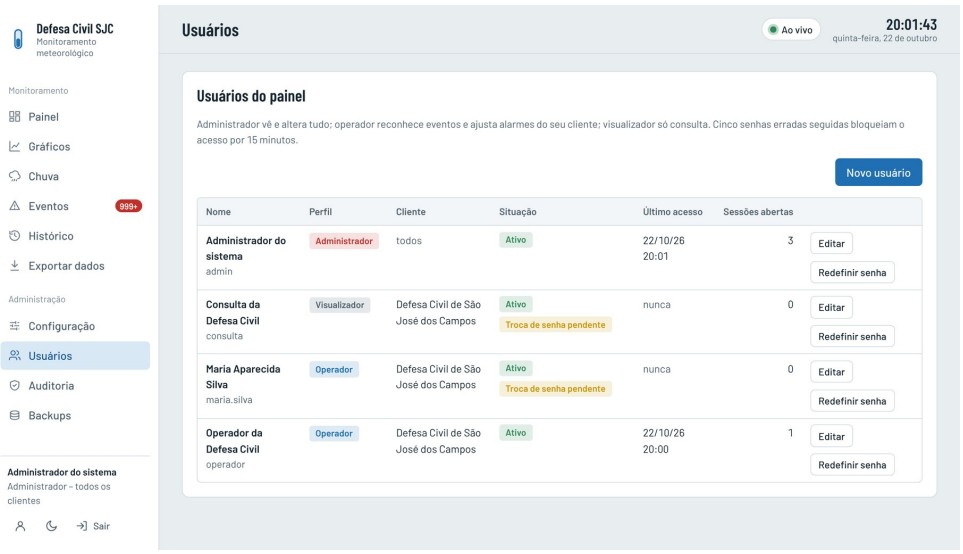
Figura 10 – Configuração das regras de alarme



Fonte: Elaborado pelos autores (2026).

Na tela de usuários (Figura 11), o administrador cadastra pessoas com perfil e cliente, redefine senhas, desativa e desbloqueia contas. A senha provisória é gerada aleatoriamente e precisa ser trocada no primeiro acesso.

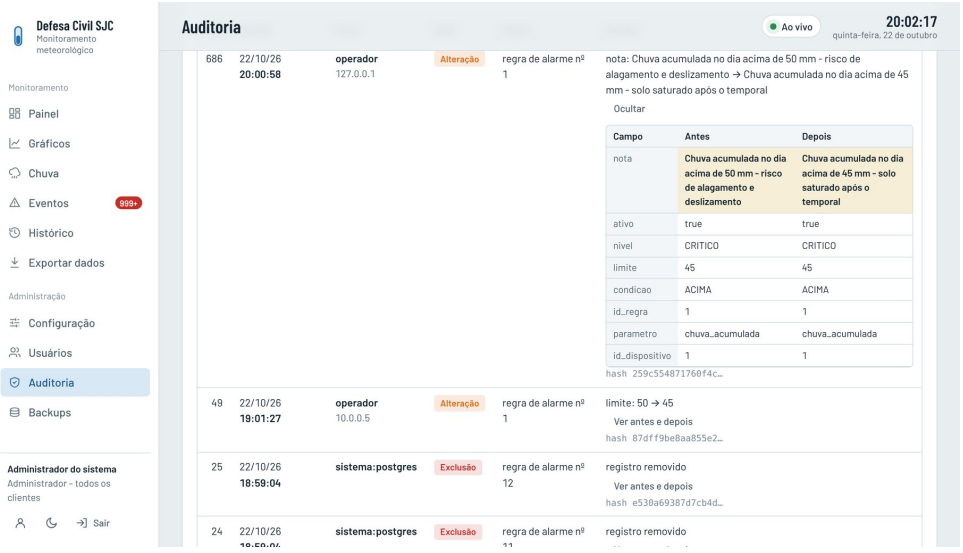
Figura 11 – Gestão de usuários



Fonte: Elaborado pelos autores (2026).

A tela de auditoria (Figura 12) verifica a corrente de hashes ao abrir e lista cada ação com usuário, IP, objeto e detalhe. Na figura, o registro 49 mostra o operador alterando o limite de chuva da Estação Centro de 50 para 45 mm, e o registro 686, outra alteração da mesma regra, aberta com o antes e depois de cada campo.

Figura 12 – Auditoria com antes e depois de uma alteração

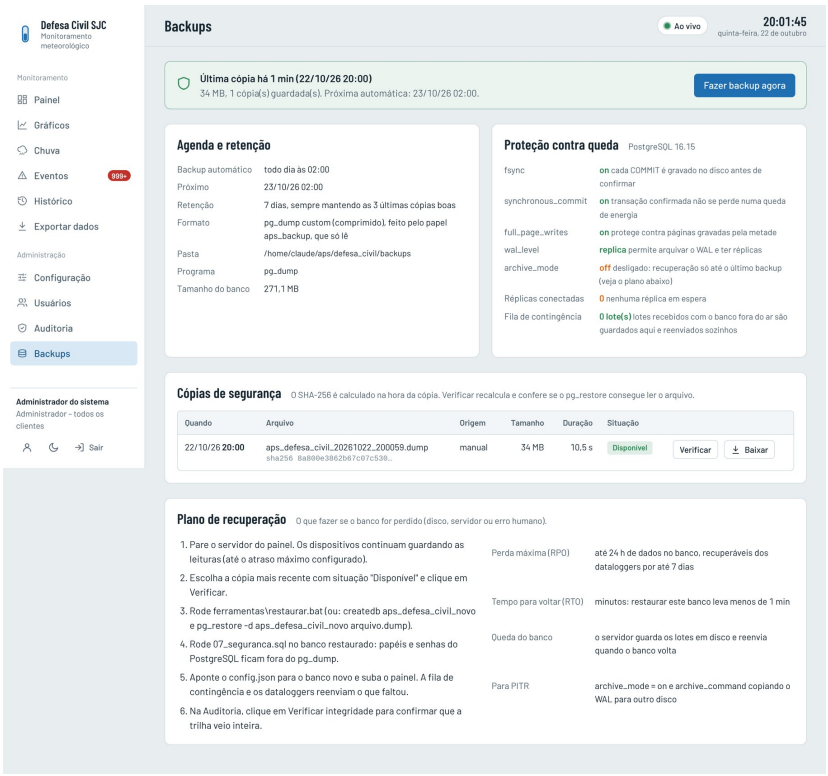


Fonte: Elaborado pelos autores (2026).

A tela de backups (Figura 13) mostra a última cópia, a próxima automática, os parâmetros de durabilidade do PostgreSQL, a fila de contingência e o plano de recuperação. Cada cópia pode ser verificada e baixada; ambas as ações ficam na auditoria.

Por fim, a tela de histórico (Figura 14) mostra o ciclo de vida dos dados: os meses arquivados, os ativos e as partições já criadas para os meses seguintes, além do resumo

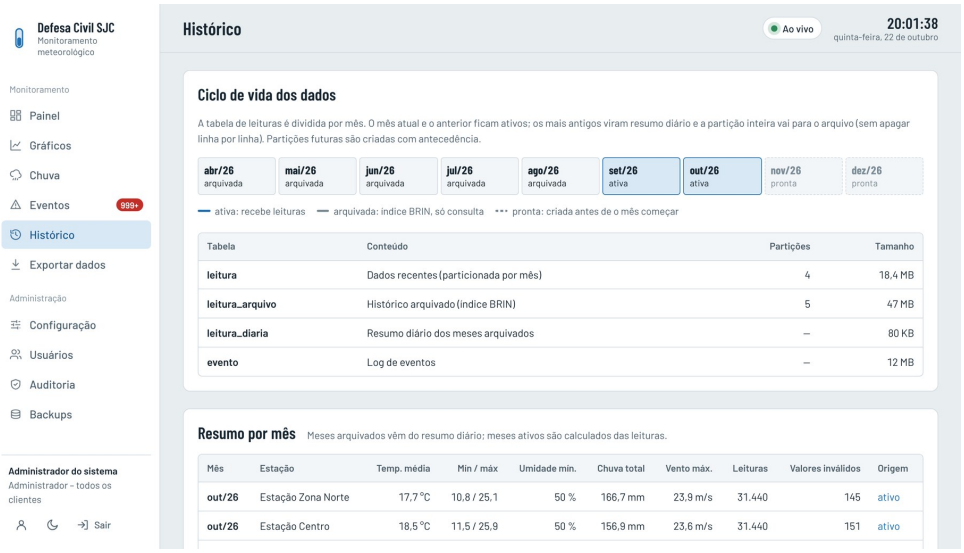
Figura 13 – Backups e continuidade



Fonte: Elaborado pelos autores (2026).

mensal de cada estação.

Figura 14 – Histórico e ciclo de vida dos dados



Fonte: Elaborado pelos autores (2026).

6 Por que o protótipo atende a dados críticos

Um sistema serve a dados críticos quando cada uma das propriedades da seção 2.4 (integridade, confidencialidade, disponibilidade e rastreabilidade) é garantida por um mecanismo concreto, e quando esse mecanismo resiste a quem tenta contorná-lo. O Quadro 7 relaciona cada ameaça ao mecanismo implementado e ao teste que comprova seu funcionamento; os testes estão detalhados no Capítulo 7.

Quadro 7 – Garantias de dados críticos no protótipo

Propriedade	**Ameaça**	**Mecanismo**	**Teste**
Integridade	Leitura com data errada no relógio do equipamento	Validação por leitura contra o relógio do servidor; rejeição registrada, sem descartar o lote	S08, S07
	Aplicação grava dado com data escolhida por ela	Função pública sem parâmetro de data; versão com data negada aos papéis da aplicação	S06
	Valor inválido do sensor (-9999) contamina médias	Faixa física por sensor; valor anulado e evento registrado	Carga
	Alteração de evento já registrado	Evento imutável; só o reconhecimento muda, com data e usuário do banco	S10
Confidencialidade	Servidor web invadido	Papéis de mínimo privilégio; banco valida o *token* da sessão a cada transação	S01, S02
	Usuário vê dados de outro cliente	RLS por cliente em todas as tabelas de dados	S01, S03
	Senha descoberta por força bruta	bcrypt; bloqueio de 15 min após 5 erros; sessão só como hash	S11
Rastreabilidade	Limite de alarme elevado sem deixar rastro	Auditoria com antes e depois, usuário, IP e data do servidor	S04
	Apagar ou alterar a própria auditoria	Trilha somente de inclusão, encadeada por SHA-256; DDL também auditado	S12, S13
Disponibilidade	Virada do mês sem partição	Criação automática das partições futuras	S09
	Banco fora do ar	Fila de contingência em disco com reenvio automático	C1
	Perda do servidor ou erro humano	Backup diário verificado por SHA-256, retenção e restauração testada	C2, C3
	Queda de energia	WAL com `fsync` e `synchronous_commit` ligados	Config.

Fonte: Elaborado pelos autores (2026).

Quatro decisões de projeto explicam por que essas garantias são confiáveis, e não apenas declaradas.

As regras ficam no banco. Validação, eventos, perfis e auditoria são implementados em PL/pgSQL, com *triggers* e políticas que valem para qualquer cliente do banco: o painel novo, o painel antigo em Streamlit, um script ou uma ferramenta de administração. Isso segue a ideia de que o SGBD deve concentrar as restrições de integridade e de autorização (Elmasri; Navathe, 2018). Os papéis da aplicação nem sequer têm permissão de inclusão

direta nas tabelas de leituras e eventos; só podem chamar as funções que aplicam as regras.

O banco não confia na aplicação. Em vez de receber da aplicação o número do usuário, o que permitiria a um servidor comprometido se passar por qualquer pessoa, o banco recebe o *token* da sessão e o valida na tabela de sessões, onde só o hash está guardado. O isolamento por cliente usa o mesmo mecanismo. Assim, o servidor web é tratado como uma camada potencialmente hostil, aplicando o mínimo privilégio de Saltzer e Schroeder (1975) também entre aplicação e banco.

O tempo é do servidor. Toda data de registro (recebimento, reconhecimento, auditoria, sessão, configuração e backup) vem do relógio do servidor, e as datas das leituras, que vêm do equipamento, são conferidas contra ele. Em dados críticos, a ordem e o momento dos fatos fazem parte do dado: saber que o alarme de chuva foi reconhecido às 15h12, e por quem, só tem valor se ninguém puder escrever outra hora.

A rastreabilidade é verificável. Uma trilha de auditoria comum pode ser editada por quem tem acesso ao banco. A trilha encadeada por hash (Schneier; Kelsey, 1999; Haber; Stornetta, 1991) não impede um superusuário de alterar uma linha, mas torna essa alteração detectável e aponta onde ela ocorreu. O próprio ato de desligar a proteção fica registrado como comando de estrutura. Combinada com a cópia de segurança, que guarda a trilha num momento anterior, essa verificação permite reconstruir o que foi alterado.

Essas medidas também atendem ao que a LGPD exige para os dados pessoais dos usuários (nome, login e endereço IP): controle de acesso, registro das operações e proteção contra acessos não autorizados (Brasil, 2018). As senhas não são guardadas em forma recuperável, e a auditoria nunca registra a senha nem o seu hash, apenas o fato de que ela foi alterada.

7 Resultados e discussão

Os testes foram executados em Linux (Ubuntu 24.04), com PostgreSQL 16.15 e Node.js 22, sempre sobre um banco recriado do zero.

7.1 Carga e detecção de eventos

A carga gerou 529.250 leituras de seis meses, com todas as validações ativas. O arquivamento manteve os dois meses mais recentes na tabela principal (149.090 leituras), moveu os anteriores para o arquivo (380.160) e gerou 264 dias de resumo. Foram registrados, nas estações Centro e Zona Norte, respectivamente, 25.928 e 25.955 transmissões OK, 523 e 501 falhas de transmissão, 1.280 e 1.358 falhas de sensor e 16 e 17 alarmes, valores próximos das taxas simuladas. No dia de temporal, as duas estações geraram alarme de vento e alarme crítico de chuva, e a chuva acumulada pelo *trigger* coincidiu com a soma das leituras.

7.2 Testes de segurança e continuidade

Os testes de segurança (Quadro 8) usaram o papel do servidor web (`aps_app`), como faria um invasor que tivesse tomado o servidor, e a API do painel. Todos tiveram o resultado esperado; o S12 mostra que nem o superusuário altera o passado sem ser percebido.

Quadro 8 – Testes de segurança e integridade

ID	Tentativa	Resultado
S01	Consultar dados sem sessão	Nenhuma linha visível (RLS)
S02	Usar *token* de sessão falso	Nenhuma linha; ações recusadas
S03	Operador lê usuários e auditoria	Permissão negada; zero linhas
S04	Operador altera limite de alarme (permitido)	Gravado e auditado com antes, depois, usuário e IP
S05	Operador altera parâmetro de administrador	Nenhuma linha afetada
S06	Aplicação grava leitura direto, apaga evento ou informa data	Permissão negada nas três
S07	Superusuário inclui leitura com data no futuro	Recusada pelo *trigger*
S08	Lote com seis leituras, cinco com datas inválidas	1 gravada; 5 rejeitadas em 3 eventos por motivo
S09	Virada de mês sem partição	Manutenção cria o mês atual e os dois seguintes
S10	Alterar evento ou desfazer reconhecimento	Recusado: evento imutável
S11	Senha errada repetidas vezes	Bloqueio de 15 min na 5ª tentativa
S12	Superusuário desliga a proteção e altera a auditoria	Verificação aponta o registro exato; o desligamento fica registrado
S13	Apagar registro da auditoria	Recusado: somente inclusão
S14	Requisição sem o cabeçalho do painel (CSRF)	Recusada (HTTP 403)
S15	Reenviar lote já gravado com -9999	Nenhum evento novo (antes, falha de sensor)

Fonte: Elaborado pelos autores (2026).

Quadro 9 – Testes de continuidade

ID	Cenário	Resultado
C1	Banco desligado durante envios	Lotes aceitos (HTTP 202) na fila e reenviados em segundos ao religar; painel avisou a queda
C2	Backup sob demanda e verificação	35,6 MB em 10,5 s; SHA-256 conferido; 450 objetos legíveis
C3	Restauração em banco novo	Mesmas contagens (149.162, 380.160 e 55.547) e auditoria íntegra
C4	Servidor parado por uma hora	Leituras recuperadas; falha de transmissão de 59 min registrada

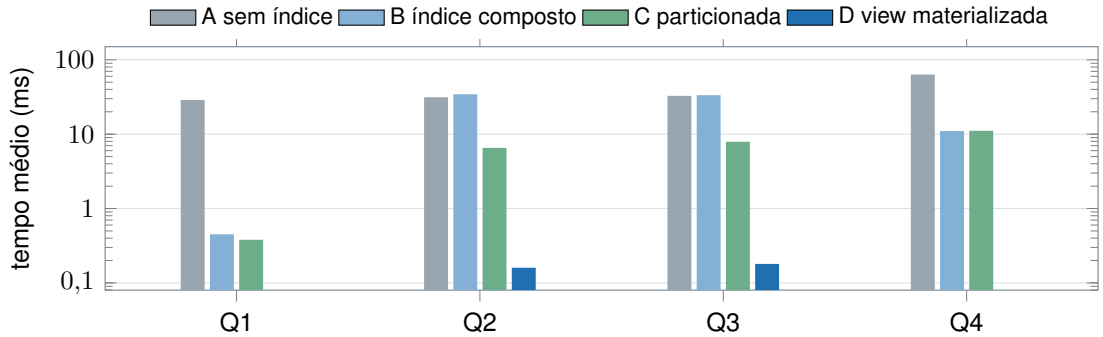
Fonte: Elaborado pelos autores (2026).

Com backup diário, a perda máxima no banco (RPO) é de 24 horas, mas na prática é menor, pois as estações guardam as leituras e o banco aceita reenvios de até sete dias; a restauração (RTO) leva minutos. O arquivamento contínuo do WAL reduziria o RPO a minutos (PostgreSQL Global Development Group, 2025a).

7.3 Desempenho e custo das validações

Na escrita de 1.000 pontos (média de 20 repetições), comparando a primeira versão e a final, o envio de um ponto por vez passou de 946,8 para 1.245,8 ms ($+31{,}6\,\%$), e o envio em lotes de dez, de 226,9 para 235,3 ms ($+3{,}7\,\%$). As validações são feitas uma vez por lote, por isso o custo se dilui no envio em lotes, modo de operação das estações, 5,3 vezes mais rápido que o unitário, como observado por Grzesik e Mrozek (2020).

Figura 15 – Tempo médio das consultas por cenário (500 repetições, escala logarítmica)



Fonte: Elaborada pelos autores (2026). Q1: uma estação em 1 dia; Q2 e Q3: médias, mínimos e máximos de 7 dias; Q4: vento por dia em 30 dias.

Nas consultas (Figura 15), o índice composto foi o melhor para uma estação (Q1, 64 vezes mais rápida), a view materializada foi de 185 a 200 vezes mais rápida nas agregações de vários dias (Q2 e Q3) e o particionamento reduziu o tempo de 4 a 6 vezes. Ler a chuva acumulada foi 4,3 vezes mais rápido que somar as leituras do dia. No PostgreSQL 18, o índice composto também acelerou Q2 e Q3 de 5 a 6,6 vezes, graças ao *skip scan* (PostgreSQL Global Development Group, 2025c).

8 Conclusão

O trabalho atingiu o objetivo de projetar e prototipar um banco de dados de telemetria IoT adequado a dados críticos. A primeira versão já recebia leituras em volume e gerava alarmes, mas não protegia o dado contra perda, adulteração e acesso indevido; as melhorias fecharam essas lacunas e foram comprovadas por testes.

Três conclusões se destacam. Primeiro, **as garantias de um dado crítico devem ficar no banco**: validação, imutabilidade, perfis, isolamento por cliente e auditoria valem para qualquer programa, e o servidor web tratado como invadido não leu nada sem um *token* válido. Segundo, **rastreabilidade só vale se for verificável**: a auditoria encadeada por hash apontou o registro exato alterado pelo próprio superusuário. Terceiro, **segurança tem custo mensurável e controlável**: menos de 4 % no envio em lotes, com as consultas mantendo ganhos de até 200 vezes.

Para a Defesa Civil, cada leitura chega validada e com data confiável, uma estação que para de transmitir é identificada, uma chuva acima do limite gera alarme no mesmo minuto, e nada se perde se o banco cair por algumas horas. As principais limitações são os dados simulados e de pequeno volume, o formato largo, que exige alterar a tabela para um novo tipo de sensor, e os testes em um único computador. Como trabalhos futuros, ficam: ativar o arquivamento do WAL e uma réplica; publicar o último hash da auditoria fora do servidor; usar HTTPS e autenticação em dois fatores; conectar uma estação real; e enviar alertas automáticos à população e às equipes.

Referências

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR ISO/IEC 27001**: segurança da informação, segurança cibernética e proteção à privacidade: sistemas de gestão da segurança da informação: requisitos. Rio de Janeiro, 2022a.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR ISO/IEC 27002**: segurança da informação, segurança cibernética e proteção à privacidade: controles de segurança da informação. Rio de Janeiro, 2022b.

BOOCH, Grady; RUMBAUGH, James; JACOBSON, Ivar. **UML**: guia do usuário. 2. ed. rev. e atual. Rio de Janeiro: Elsevier, 2006.

BRASIL. Lei nº 12.608, de 10 de abril de 2012. Institui a Política Nacional de Proteção e Defesa Civil – PNPDEC; dispõe sobre o Sistema Nacional de Proteção e Defesa Civil – SINPDEC e o Conselho Nacional de Proteção e Defesa Civil – CONPDEC; autoriza a criação de sistema de informações e monitoramento de desastres. **Diário Oficial da União:** seção 1, Brasília, DF, p. 1, 11 abr. 2012. Disponível em: <https://www.planalto.gov.br/ccivil_03/_ato2011-2014/2012/lei/l12608.htm>. Acesso em: 22 out. 2026.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Dispõe sobre a proteção de dados pessoais e altera a Lei nº 12.965, de 23 de abril de 2014 (Marco Civil da Internet). **Diário Oficial da União:** seção 1, Brasília, DF, p. 59, 15 ago. 2018. Disponível em: <https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm>. Acesso em: 22 out. 2026.

CENTRO NACIONAL DE MONITORAMENTO E ALERTAS DE DESASTRES NATURAIS. **Cemaden amplia a lista de municípios monitorados, de 1038 para 1133**. gov.br. 26 mar. 2024. Disponível em: <https://www.gov.br/cemaden/pt-br/cemaden-amplia-a-lista-de-municipios-monitorados-de-1038-para-1133>. Acesso em: 22 out. 2026.

CHIRKOVA, Rada; YANG, Jun. Materialized views. **Foundations and Trends in Databases**, Hanover, v. 4, n. 4, p. 295–405, 2012. DOI: https://doi.org/10.1561/1900000020.

CODD, E. F. A relational model of data for large shared data banks. **Communications of the ACM**, New York, v. 13, n. 6, p. 377–387, 1970. DOI: https://doi.org/10.1145/362384.362685.

ELMASRI, Ramez; NAVATHE, Shamkant B. **Sistemas de banco de dados**. 7. ed. São Paulo: Pearson, 2018.

GRZESIK, Piotr; MROZEK, Dariusz. Comparative analysis of time series databases in the context of edge computing for low power sensor networks. In: KRZHIZHANOVSKAYA, Valeria V. *et al.* (Ed.). **Computational Science – ICCS 2020**. Cham: Springer, 2020. v. 12141. (Lecture Notes in Computer Science), p. 371–383. DOI: https://doi.org/10.1007/978-3-030-50426-7_28.

HABER, Stuart; STORNETTA, W. Scott. How to time-stamp a digital document. **Journal of Cryptology**, New York, v. 3, n. 2, p. 99–111, 1991. DOI: https://doi.org/10.1007/BF00196791.

HÄRDER, Theo; REUTER, Andreas. Principles of transaction-oriented database recovery. **ACM Computing Surveys**, New York, v. 15, n. 4, p. 287–317, 1983. DOI: https://doi.org/10.1145/289.291.

JENSEN, Søren Kejser; PEDERSEN, Torben Bach; THOMSEN, Christian. Time series management systems: a survey. **IEEE Transactions on Knowledge and Data Engineering**, Piscataway, v. 29, n. 11, p. 2581–2600, 2017. DOI: https://doi.org/10.1109/TKDE.2017.2740932.

MDN WEB DOCS. **Using server-sent events**. 2026. Disponível em: <https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events/Using_server-sent_events>. Acesso em: 22 out. 2026.

MOHAN, C. *et al.* ARIES: a transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging. **ACM Transactions on Database Systems**, New York, v. 17, n. 1, p. 94–162, 1992. DOI: https://doi.org/10.1145/128765.128770.

NACHTIGALL, Luiz F. **Chuva de quase 700 mm inunda litoral de São Paulo e vai seguir**. MetSul Meteorologia. 19 fev. 2023. Disponível em: <https://metsul.com/chuva-de-quase-700-mm-inunda-litoral-de-sao-paulo-e-vai-seguir/>. Acesso em: 22 out. 2026.

NAKANO, Anderson Kazuo; MARU, Henry Tomio Kreniski. O planejamento urbano e a gestão de riscos em municípios monitorados pelo Cemaden. **Boletim Regional, Urbano e Ambiental**, Brasília, n. 33, p. 67–77, dez. 2024. DOI: https://doi.org/10.38116/brua33art5.

OPENJS FOUNDATION. **Node.js documentation**. 2026. Disponível em: <https://nodejs.org/docs/latest/api/>. Acesso em: 22 out. 2026.

OWASP FOUNDATION. **OWASP Top 10:2021**. 2021. Disponível em: <https://owasp.org/Top10/2021/>. Acesso em: 22 out. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **Continuous archiving and point-in-time recovery (PITR)**: PostgreSQL 18 documentation, seção 25.3. 2025a. Disponível em: <https://www.postgresql.org/docs/18/continuous-archiving.html>. Acesso em: 22 out. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 18 documentation**. 2025b. Disponível em: <https://www.postgresql.org/docs/18/>. Acesso em: 22 out. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **Release 18**: PostgreSQL 18 documentation, apêndice E. 2025c. Disponível em: <https://www.postgresql.org/docs/18/release-18.html>. Acesso em: 22 out. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **Row security policies**: PostgreSQL 18 documentation, seção 5.9. 2025d. Disponível em: <https://www.postgresql.org/docs/18/ddl-rowsecurity.html>. Acesso em: 22 out. 2026.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de software**: uma abordagem profissional. 9. ed. Porto Alegre: AMGH, 2021.

PROVOS, Niels; MAZIÈRES, David. A future-adaptable password scheme. In: PROCEEDINGS of the FREENIX Track: 1999 USENIX Annual Technical Conference. Monterey: USENIX Association, 1999. Disponível em: <https://www.usenix.org/legacy/events/usenix99/provos/provos.pdf>. Acesso em: 22 out. 2026.

SALTZER, J. H.; SCHROEDER, M. D. The protection of information in computer systems. **Proceedings of the IEEE**, New York, v. 63, n. 9, p. 1278–1308, 1975. DOI: https://doi.org/10.1109/PROC.1975.9939.

SCHNEIER, Bruce; KELSEY, John. Secure audit logs to support computer forensics. **ACM Transactions on Information and System Security**, New York, v. 2, n. 2, p. 159–176, 1999. DOI: https://doi.org/10.1145/317087.317089.

SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. **Sistema de banco de dados**. 7. ed. Rio de Janeiro: LTC, 2020.

SOMMERVILLE, Ian. **Engenharia de software**. 10. ed. São Paulo: Pearson, 2018.

SWANSON, Marianne *et al.* **Contingency planning guide for federal information systems**. Gaithersburg, 2010. DOI: https://doi.org/10.6028/NIST.SP.800-34r1.